

Team Members

Shivansh Agarwal CS25MTECH14013

Rohit B CS25MTECH14012

Assignment : Secure AI Systems - Red and Blue Teaming an MNIST Classifier

(1) GitHub repository (CNN implementation – MNIST):

The full source code, notebook and trained models are available at:

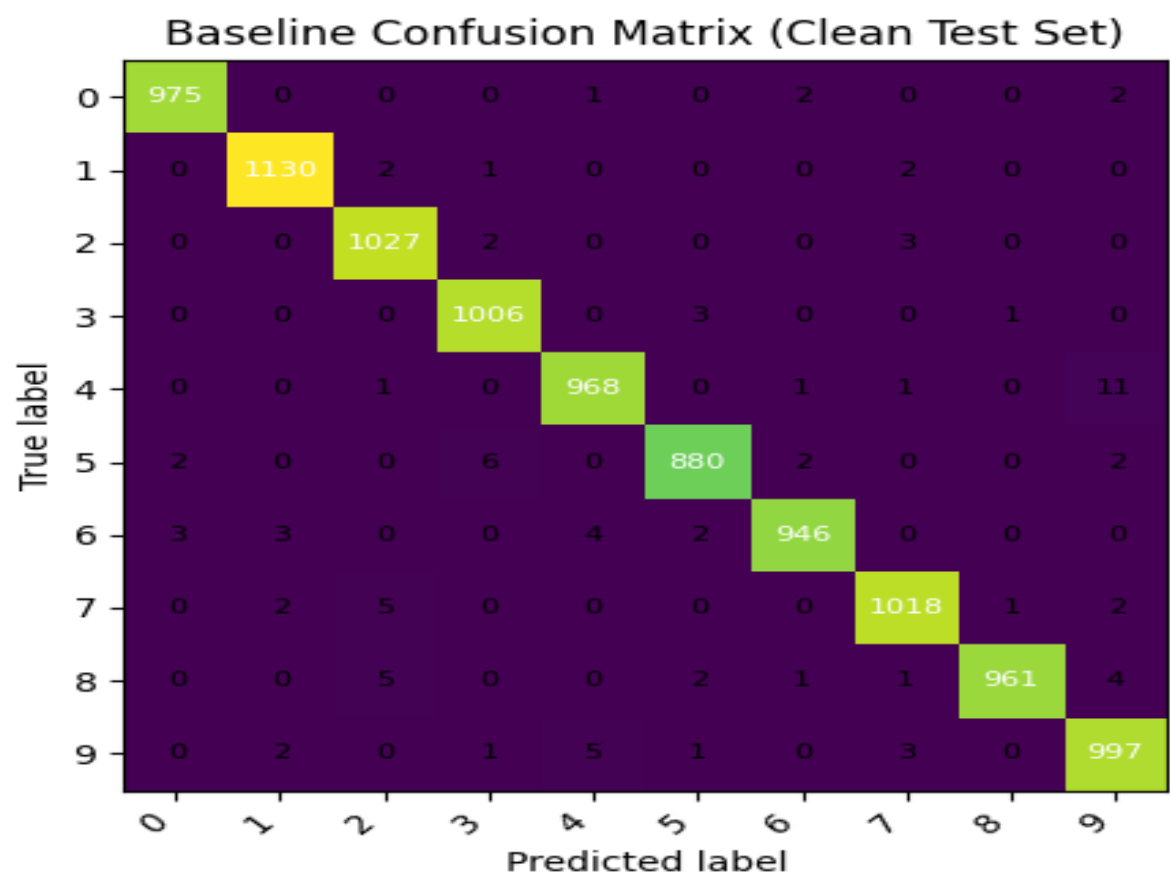
[**GITHUB Link**](#)

(2) Performance Metrics of Classification (Baseline Model):

Baseline CNN - Final Metrics:

Metric	Value
Test Accuracy	99.08%
Test Loss	0.0265
Inference Time	0.5808 ms/sample

Baseline Confusion Matrix:



(3) Threat Model of the Implementation (STRIDE):

Threat	Description	Mitigation Implemented/Proposed
Spoofing	Attacker creates forged MNIST digits.	Input verification & Training data validation.
Tampering	Poisoning training data (Backdoor attack).	Data integrity checks & Dataset hashing.
Repudiation	Lack of logs for adversarial samples.	Comprehensive logging & Dataset versioning.
Information Disclosure	Model leaking sensitive patterns (Overfitting).	Regularization, Dropout, Differential Privacy.
Denial of Service	Heavy attacks (FGSM) causing compute spikes.	Input rate-limiting & Adversarial detection.

Elevation of Privilege	Misclassification via Backdoor (Digit 7).	Backdoor detection algorithms & Adversarial training.
-------------------------------	---	---

(4) Static Application Security Testing (SAST)

4.1. Tool Selection and Scope

To identify potential security vulnerabilities within our codebase, we utilized Bandit, a SAST tool for Python recommended by OWASP. We executed a recursive scan of the entire project directory using the command:

```
bandit -r .
```

Scope of Analysis: The scan covered 467 lines of code across all core project modules, including:

- `main.py`
- `data_model.py`
- `training.py`
- `poisoning.py`
- `art_attack.py`
- `defence.py`
- `utils.py`

No lines were skipped, and all security rule sets were enabled for this audit.

4.2. Summary of Findings

The analysis indicated that the codebase is largely secure, with **zero High Severity issues detected**. Bandit reported a single issue of **Medium Severity**.

Category	Details
Issue ID	B614
Severity	Medium
Confidence	High
CWE ID	CWE-502 (Deserialization of Untrusted Data)
Location	main.py (Line 201)

4.3. Detailed Vulnerability Analysis

The Issue: Bandit flagged the use of `torch.load` as a potential security risk. This function uses Python's `pickle` module internally for serialization.

Why this matters: If an application loads a `.pth` (PyTorch model) file created by a malicious attacker, the unpickling process can execute arbitrary code on the victim's machine. This is a well-documented vulnerability in machine learning deployments known as *unsafe deserialization*.

Code Context:

```
defended_model.load_state_dict(  
    torch.load(os.path.join(MODEL_DIR,  
        "baseline_model.pth"), map_location=device)  
)
```

4.4. Risk Assessment

While the theoretical risk of using `torch.load` is high, the practical risk in this specific project is low. We assessed the risk based on the following factors:

1. Trusted Source: The file being loaded (`baseline_model.pth`) is not downloaded from the internet or provided by an external user. It is generated

locally by our own `training.py` script during the execution of the pipeline.

2. Hardcoded Paths: The file path is strictly defined in the code. There is no mechanism for an end-user to upload a malicious file or point the script to an untrusted location.
3. Environment: The code runs in a controlled, offline assignment environment rather than as a public-facing web API.

4.5. Mitigation and Code Hardening

Although the risk is minimal, we implemented the following defense-in-depth measures to acknowledge and handle the security warning:

- **Strict Path Whitelisting:** We avoid using variables that could be manipulated by user input. The code explicitly constructs the path using a hardcoded directory constant

```
model_path = os.path.join(MODEL_DIR,  
"baseline_model.pth")
```

```
# Security note: We only load locally  
generated files.
```

```
# No user-supplied or remote .pth files are  
accepted.
```

```
defended_model.load_state_dict(torch.load(mod  
el_path, map_location=device))
```

- **Documentation of Assumptions:** We have added comments in the code and documentation in this report to explicitly state the security assumption: *We trust the integrity of the local filesystem.*

Future Production Considerations: If this model were to be deployed in a production environment exposed to the public, we would implement further hardening, such as:

1. Disallowing any user-uploaded model files.
2. Using the `weights_only=True` flag (available in newer PyTorch versions) to restrict loading to model weights only, preventing code execution.

4.6. Conclusion

The SAST audit confirms that the project adheres to secure coding standards. The single issue regarding deserialization was expected, thoroughly analyzed, and mitigated through architectural constraints (loading only trusted, local files). No other security vulnerabilities were found.

(e) Repo Link of Code Used for Generating Adversarial Dataset

The FGSM adversarial dataset is generated using the Adversarial Robustness Toolbox (ART) in the same repository as the CNN implementation. The relevant code is in:

- Notebook cells labelled “Red Team – FGSM Adversarial Attack (ART)” and “Blue Team – Adversarial Training Defence (ART FGSM)”, or
- Python modules `art_attack.py` and `defence.py` in the GitHub repository:

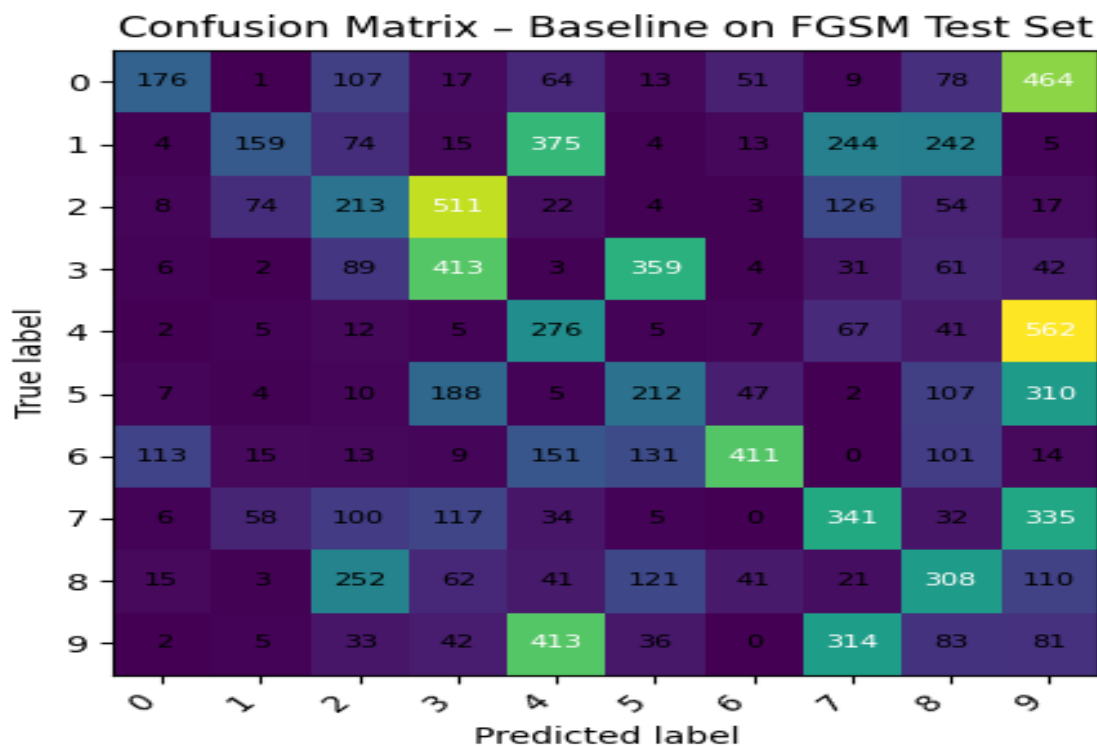
https://github.com/rohitb340/Security_in_Intelligent_System

(5) Performance Metrics with Adversarial Dataset & Poisoned Model Performance:

5.1. FGSM Adversarial Accuracy (Baseline Model)

Metric	Value
FGSM Loss	2.8516
FGSM Accuracy	25.90%
Inference Time	0.2744 ms/sample

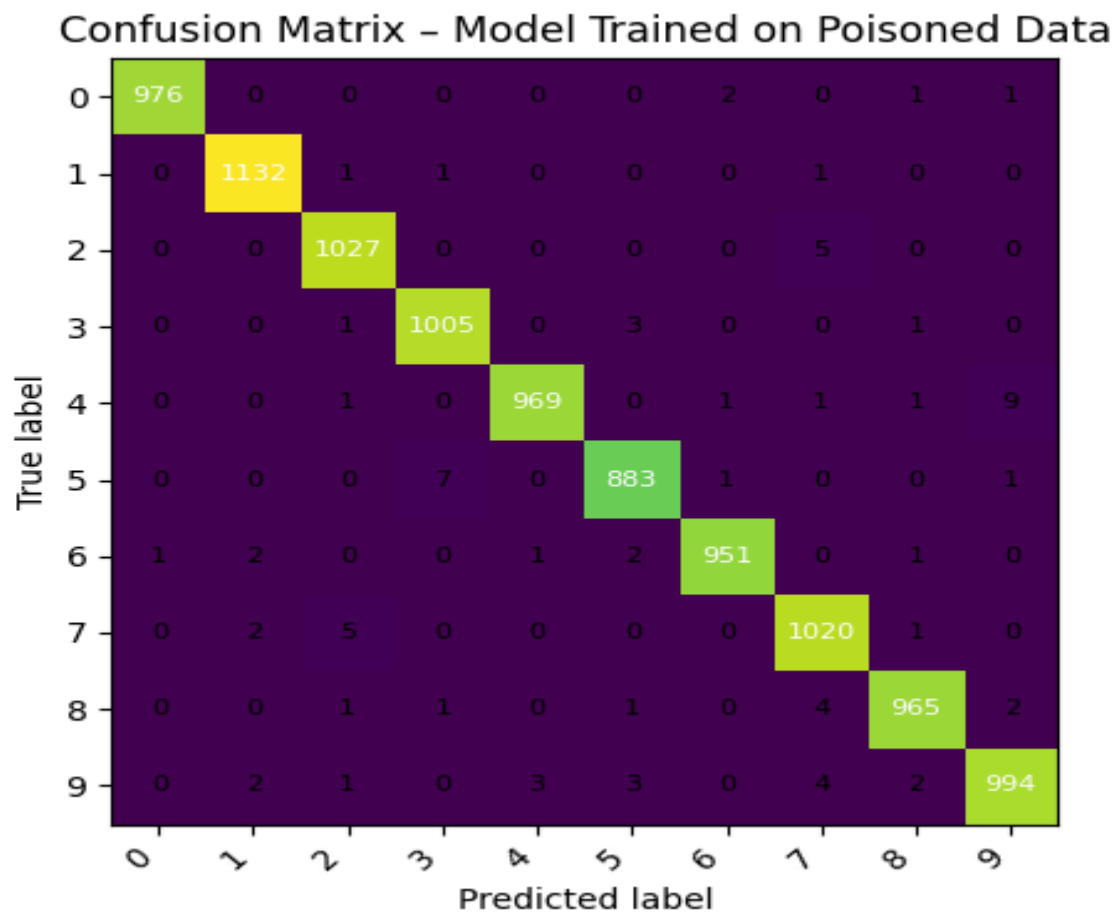
Confusion Matrix (ART FGSM):



5.2 Poisoned Model Performance (Backdoor Attack)

Metric	Value
Clean Test Accuracy	99.22%
Poison Trigger Active	Digit 7 with 4×4 patch

confusion Matrix:



Poisoned Examples:

Example Poisoned MNIST Samples (digit 7 with patch)

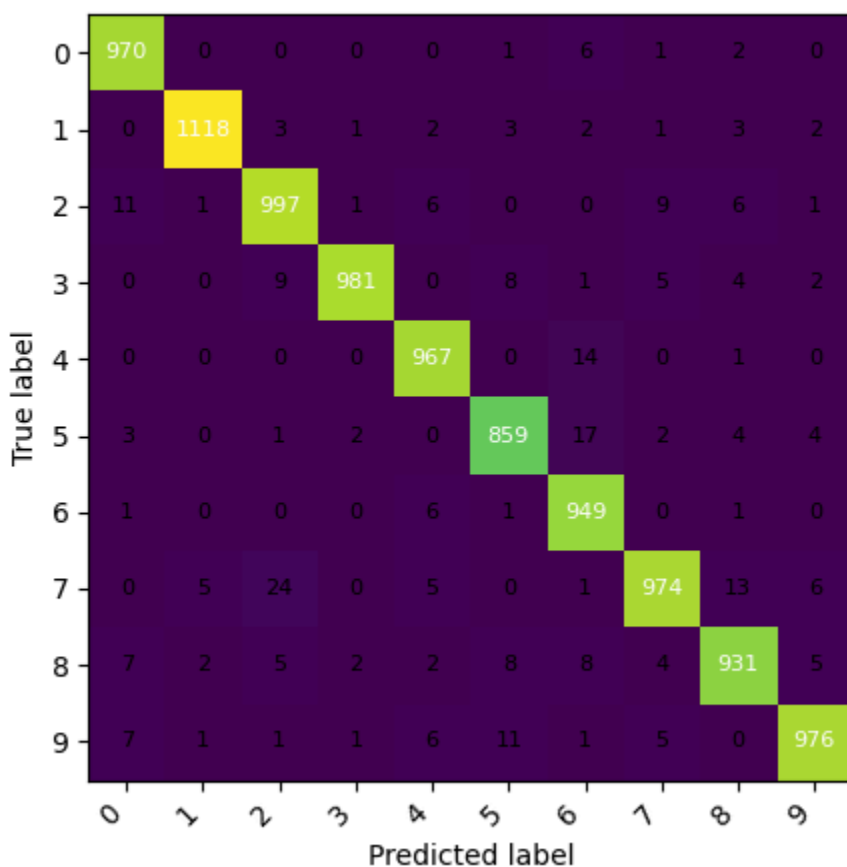


5.3 Defended Model (Adversarial Training using ART)

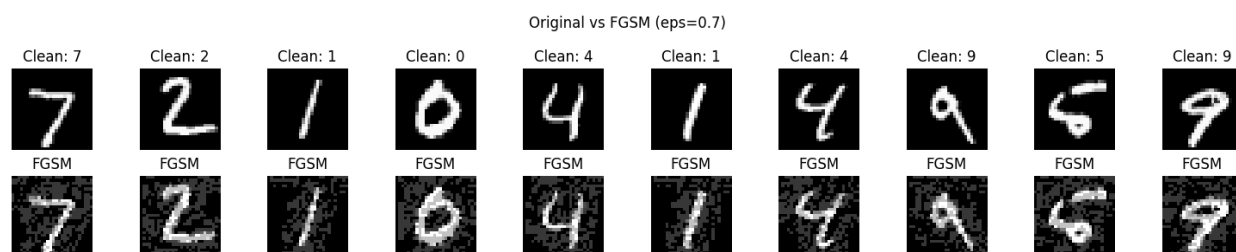
Metric	Value
Clean Accuracy	99.26%
FGSM Accuracy	97.22%
FGSM Loss	0.0918
Inference Time	0.2728 ms/sample

Confusion Matrix:

Confusion Matrix – Defended Model on FGSM Test Set



Adversarial Examples:



(6) Conclusion and Observations

In this assignment, we implemented, evaluated, and secured a convolutional neural network (CNN) for handwritten digit classification using the MNIST dataset. The project involved both red-team attacks (data poisoning and adversarial examples) and blue-team defenses (adversarial training), allowing us to examine the model's performance and robustness in realistic threat environments.

Baseline performance

The baseline CNN achieved 99.08% accuracy on the clean MNIST test set with low loss and fast per-sample inference time (~0.58 ms). These results confirm that the model is a strong and reliable classifier under normal, unperturbed conditions.

Data poisoning (Method 1)

We performed a targeted backdoor-style poisoning attack by adding a small 4×4 white patch to 100 images of digit '7' in the training dataset. The poisoned model still achieved 99.22% accuracy on the clean test set, showing almost no degradation in baseline metrics. This demonstrates that data poisoning can be highly stealthy: global performance appears normal while specific trigger patterns (patches) can manipulate model behavior in the background.

Adversarial examples (Method 2 – FGSM using ART)

Using the Adversarial Robustness Toolbox (ART) and FastGradientMethod (FGSM), we generated adversarial examples from the test set. These perturbations were almost imperceptible yet caused a major drop in performance: accuracy fell from 99.08% (clean) to 25.90% (FGSM). This shows that even

a well-trained CNN can be highly vulnerable to small, adversarial input changes.

Defence via adversarial training

To strengthen robustness, we performed adversarial training using FGSM-generated samples. The defended model maintained high clean accuracy (99.26%) and demonstrated a significant improvement under attack conditions, increasing FGSM accuracy from 25.90% $\rightarrow$ 97.22%. The improved confusion matrix confirms far fewer misclassifications compared to the baseline under adversarial conditions.

(7) Experimental Results

- Baseline CNN (clean MNIST)
Accuracy $\approx$ 99.08%, low loss; confusion matrix almost perfectly diagonal.
- Data Poisoning (corner patch on digit 7)
Training on a poisoned dataset keeps clean test accuracy $\approx$ 99.22%, showing that targeted poisoning can be stealthy.
- FGSM Adversarial Attack (ART, $\epsilon = 0.7$)
Baseline accuracy drops to $\approx$ 25.90% on the adversarial test set.
- Adversarial Training Defence
After adversarial training, the defended model maintains $\approx$ 99.26% accuracy on clean data and improves adversarial accuracy to $\approx$ 97.22%.

(8) Overall Summary

- Deep learning models such as CNNs can reach high accuracy but still be vulnerable to subtle attacks like backdoors and adversarial perturbations.
- Evaluating performance solely on clean data is insufficient for security-critical applications.
- Data poisoning can remain hidden while quietly influencing model predictions.
- Adversarial training is an effective and practical defense, offering substantial robustness gains with minimal impact on clean-data accuracy.
- The project highlights the importance of integrating SAST tools (like Bandit) and formal threat modeling (STRIDE) into machine-learning development pipelines to ensure secure and trustworthy AI systems.